

Exercise 6

All based on Games using Heuristic

1. Execute below given python code and comment each and every line to describe its logic and functionality. (TicTac Toe)

```
# -*- coding: utf-8 -*-
```

```
"""
```

```
Created on Sun Aug 30 22:45:27 2026
```

```
@author: admin
```

```
"""
```

```
import tkinter as tk
```

```
from tkinter import messagebox
```

```
import random
```

```
class TicTacToe:
```

```
    def __init__(self, root):
```

```
        self.root = root
```

```
        self.root.title("Tic Tac Toe")
```

```
        self.root.geometry("400x500")
```

```
        self.board = [""] * 9
```

```
self.buttons = []
```

```
self.player = "X"
```

```
self.computer = "O"
```

```
# Title
```

```
title = tk.Label(  
    root,  
    text="TIC TAC TOE",  
    font=("Arial", 24, "bold")  
)
```

```
title.pack(pady=20)
```

```
# Game board
```

```
board_frame = tk.Frame(root)
```

```
board_frame.pack()
```

```
for i in range(9):
```

```
    button = tk.Button(  
        board_frame,  
        text="",  
        font=("Arial", 30, "bold"),  
        width=4,
```

```
        height=2,  
        command=lambda i=i: self.player_move(i)  
    )
```

```
    button.grid(  
        row=i // 3,  
        column=i % 3,  
        padx=3,  
        pady=3  
    )
```

```
    self.buttons.append(button)
```

```
# Status
```

```
self.status = tk.Label(  
    root,  
    text="Your turn (X)",  
    font=("Arial", 14)  
)  
self.status.pack(pady=15)
```

```
# New Game button
```

```
new_game = tk.Button(
```

```

        root,
        text="New Game",
        font=("Arial", 12),
        command=self.reset_game
    )
    new_game.pack()

# -----
# Player move
# -----

def player_move(self, position):

    # Ignore occupied cell
    if self.board[position] != "":
        return

    # Put X
    self.board[position] = self.player
    self.buttons[position]["text"] = self.player

    # Check player win
    if self.check_winner(self.player):

```

```
self.status["text"] = "You Win!"  
  
messagebox.showinfo("Game Over", "Congratulations! You Win!")  
  
self.disable_board()  
  
return
```

```
# Check draw
```

```
if "" not in self.board:
```

```
    self.status["text"] = "Draw!"  
  
    messagebox.showinfo("Game Over", "Game Draw!")  
  
    return
```

```
# Computer turn
```

```
self.status["text"] = "Computer thinking..."
```

```
self.root.after(500, self.computer_move)
```

```
# -----
```

```
# Computer move
```

```
# -----
```

```
def computer_move(self):
```

```
    position = self.find_best_move()
```

```
if position is not None:
```

```
    self.board[position] = self.computer
```

```
    self.buttons[position]["text"] = self.computer
```

```
# Check computer win
```

```
if self.check_winner(self.computer):
```

```
    self.status["text"] = "Computer Wins!"
```

```
    messagebox.showinfo(
```

```
        "Game Over",
```

```
        "Computer Wins!"
```

```
    )
```

```
    self.disable_board()
```

```
    return
```

```
# Check draw
```

```
if "" not in self.board:
```

```
    self.status["text"] = "Draw!"
```

```
    messagebox.showinfo(
```

```
        "Game Over",
```

```
        "Game Draw!"
```

```
    )
```

```
return
```

```
self.status["text"] = "Your turn (X)"
```

```
# -----
```

```
# Simple heuristic AI
```

```
# -----
```

```
def find_best_move(self):
```

```
    # 1. Try to WIN
```

```
    for i in range(9):
```

```
        if self.board[i] == "":
```

```
            self.board[i] = self.computer
```

```
            if self.check_winner(self.computer):
```

```
                self.board[i] = ""
```

```
                return i
```

```
            self.board[i] = ""
```

2. BLOCK player's winning move

for i in range(9):

if self.board[i] == "":

self.board[i] = self.player

if self.check_winner(self.player):

self.board[i] = ""

return i

self.board[i] = ""

3. Take CENTER

if self.board[4] == "":

return 4

4. Take a CORNER

corners = [0, 2, 6, 8]

empty_corners = [

i for i in corners

if self.board[i] == ""

```
]
```

```
if empty_corners:
```

```
    return random.choice(empty_corners)
```

```
# 5. Take any available position
```

```
empty_positions = [
```

```
    i for i in range(9)
```

```
    if self.board[i] == ""
```

```
]
```

```
if empty_positions:
```

```
    return random.choice(empty_positions)
```

```
return None
```

```
# -----
```

```
# Check winner
```

```
# -----
```

```
def check_winner(self, player):
```

```
    winning_patterns = [
```

```
# Rows
(0, 1, 2),
(3, 4, 5),
(6, 7, 8),

# Columns
(0, 3, 6),
(1, 4, 7),
(2, 5, 8),

# Diagonals
(0, 4, 8),
(2, 4, 6)
]
```

```
for pattern in winning_patterns:
```

```
    a, b, c = pattern
```

```
    if (
```

```
        self.board[a] == player and
```

```
        self.board[b] == player and
```

```

        self.board[c] == player
    ):
        return True

    return False

# -----
# Disable board
# -----

def disable_board(self):

    for button in self.buttons:
        button["state"] = "disabled"

# -----
# Reset game
# -----

def reset_game(self):

    self.board = [""] * 9

```

```
for button in self.buttons:

    button["text"] = ""

    button["state"] = "normal"


self.status["text"] = "Your turn (X)"
```

```
# -----
# Main program
# -----
```

```
root = tk.Tk()
```

```
game = TicTacToe(root)
```

```
root.mainloop()
```

2. Execute below given python code and comment each and every line to describe its logic and functionality.(Rock-Paper-Scissors — Frequency Heuristic)

```
import tkinter as tk
```

```
import random
```

```
from collections import Counter
```

```
class RPS:
```

```
    def __init__(self, root):
```

```
        self.root = root
```

```
        self.root.title("Rock Paper Scissors")
```

```
        self.history = []
```

```
        self.player_score = 0
```

```
        self.computer_score = 0
```

```
        tk.Label(
```

```
            root,
```

```
            text="ROCK PAPER SCISSORS",
```

```
            font=("Arial", 22, "bold")
```

```
        ).pack(pady=20)
```

```
        self.result = tk.Label(
```

```
            root,
```

```
            text="Choose your move",
```

```
            font=("Arial", 15)
```

```
)
```

```
self.result.pack(pady=10)
```

```
frame = tk.Frame(root)
```

```
frame.pack()
```

```
for move in ["Rock", "Paper", "Scissors"]:
```

```
    tk.Button(
```

```
        frame,
```

```
        text=move,
```

```
        width=12,
```

```
        command=lambda m=move: self.play(m)
```

```
    ).pack(side="left", padx=5)
```

```
self.score = tk.Label(
```

```
    root,
```

```
    text="You: 0   Computer: 0",
```

```
    font=("Arial", 14)
```

```
)
```

```
self.score.pack(pady=20)
```

```
def play(self, player):

    self.history.append(player)

    computer = self.predict_move()

    winner = self.get_winner(player, computer)

    if winner == "player":
        self.player_score += 1
        text = "You Win!"

    elif winner == "computer":
        self.computer_score += 1
        text = "Computer Wins!"

    else:
        text = "Draw!"

    self.result["text"] = (
        f"You: {player}\n"
        f"Computer: {computer}\n\n"
        f"{text}"
    )
```

```
)
```

```
self.score["text"] = (  
    f"You: {self.player_score}  "  
    f"Computer: {self.computer_score}"  
)
```

```
def predict_move(self):
```

```
    # Not enough history  
    if len(self.history) < 3:  
        return random.choice(  
            ["Rock", "Paper", "Scissors"]  
        )
```

```
counts = Counter(self.history)
```

```
predicted = counts.most_common(1)[0][0]
```

```
    # Counter move
```

```
    if predicted == "Rock":  
        return "Paper"
```

```
if predicted == "Paper":
```

```
    return "Scissors"
```

```
    return "Rock"
```

```
def get_winner(self, p, c):
```

```
    if p == c:
```

```
        return "draw"
```

```
    if (
```

```
        (p == "Rock" and c == "Scissors") or
```

```
        (p == "Paper" and c == "Rock") or
```

```
        (p == "Scissors" and c == "Paper"))
```

```
    ):

```

```
        return "player"
```

```
    return "computer"
```

```
root = tk.Tk()
```

```
game = RPS(root)
```

```
root.mainloop()
```

3. Execute Below given code and Write each line with comment describing the logic and functionality. (Number Guessing Game — Hot/Cold Heuristic)

```
import tkinter as tk
```

```
from tkinter import messagebox
```

```
class NumberGame:
```

```
    def __init__(self, root):
```

```
        self.root = root
```

```
        self.root.title("Number Guessing AI")
```

```
        self.low = 1
```

```
        self.high = 100
```

```
        self.guess = None
```

```
        tk.Label(
```

```
            root,
```

```
            text="Think of a number from 1 to 100",
```

```
            font=("Arial", 18, "bold")
```

```
).pack(pady=20)
```

```
self.display = tk.Label(
```

```
    root,
```

```
    text="",
```

```
    font=("Arial", 30)
```

```
)
```

```
self.display.pack(pady=20)
```

```
frame = tk.Frame(root)
```

```
frame.pack()
```

```
tk.Button(
```

```
    frame,
```

```
    text="Higher",
```

```
    width=12,
```

```
    command=self.higher
```

```
).grid(row=0, column=0, padx=5)
```

```
tk.Button(
```

```
    frame,
```

```
    text="Correct",
```

```
    width=12,
```

```
        command=self.correct  
    ).grid(row=0, column=1, padx=5)
```

```
tk.Button(  
    frame,  
    text="Lower",  
    width=12,  
    command=self.lower  
).grid(row=0, column=2, padx=5)
```

```
tk.Button(  
    root,  
    text="New Game",  
    command=self.reset  
).pack(pady=20)
```

```
self.make_guess()
```

```
def make_guess(self):
```

```
    if self.low <= self.high:
```

```
        self.guess = (self.low + self.high) // 2
```

```
self.display["text"] = str(self.guess)
```

```
else:
```

```
    messagebox.showerror(  
        "Error",  
        "Your answers are inconsistent."  
    )
```

```
def higher(self):
```

```
    self.low = self.guess + 1  
    self.make_guess()
```

```
def lower(self):
```

```
    self.high = self.guess - 1  
    self.make_guess()
```

```
def correct(self):
```

```
    messagebox.showinfo(  
        "Correct",  
        "You have guessed the number correctly."  
    )
```

```
        "Game",  
        f"I found your number: {self.guess}"  
    )
```

```
    self.reset()
```

```
def reset(self):
```

```
    self.low = 1
```

```
    self.high = 100
```

```
    self.make_guess()
```

```
root = tk.Tk()
```

```
game = NumberGame(root)
```

```
root.mainloop()
```

4. . Execute below given python code and comment each and every line to describe its logic and functionality (Catch the Target — Movement Prediction Heuristic)

```
import tkinter as tk
```

```
import random
```

```
WIDTH = 600
```

```
HEIGHT = 400
```

```
STEP = 20
```

```
class CatchGame:
```

```
    def __init__(self, root):
```

```
        self.root = root
```

```
        self.root.title("Catch the Target")
```

```
        self.canvas = tk.Canvas(
```

```
            root,
```

```
            width=WIDTH,
```

```
        height=HEIGHT,  
        bg="white"  
    )
```

```
self.canvas.pack()
```

```
self.player_x = 100
```

```
self.player_y = 200
```

```
self.target_x = 500
```

```
self.target_y = 200
```

```
self.score = 0
```

```
self.player = self.canvas.create_rectangle(  
    self.player_x,  
    self.player_y,  
    self.player_x + 30,  
    self.player_y + 30  
)
```

```
self.target = self.canvas.create_oval(  
    self.target_x,
```

```
        self.target_y,  
        self.target_x + 25,  
        self.target_y + 25  
    )
```

```
self.info = tk.Label(  
    root,  
    text="Use arrow keys to catch the target",  
    font=("Arial", 14)  
)
```

```
self.info.pack()
```

```
root.bind("<Left>", self.left)  
root.bind("<Right>", self.right)  
root.bind("<Up>", self.up)  
root.bind("<Down>", self.down)
```

```
self.ai_move()
```

```
def left(self, event):  
    self.player_x -= STEP  
    self.update_player()
```

```
def right(self, event):  
    self.player_x += STEP  
    self.update_player()
```

```
def up(self, event):  
    self.player_y -= STEP  
    self.update_player()
```

```
def down(self, event):  
    self.player_y += STEP  
    self.update_player()
```

```
def update_player(self):  
  
    self.player_x = max(  
        0,  
        min(WIDTH - 30, self.player_x)  
    )  
  
    self.player_y = max(  
        0,  
        min(HEIGHT - 30, self.player_y)
```

)

```
self.canvas.coords(  
    self.player,  
    self.player_x,  
    self.player_y,  
    self.player_x + 30,  
    self.player_y + 30  
)
```

```
def ai_move(self):
```

```
    # Heuristic:
```

```
    # Move target toward player
```

```
    if self.target_x < self.player_x:
```

```
        self.target_x += STEP
```

```
    elif self.target_x > self.player_x:
```

```
        self.target_x -= STEP
```

```
    if self.target_y < self.player_y:
```

```
        self.target_y += STEP
```

```
elif self.target_y > self.player_y:
```

```
    self.target_y -= STEP
```

```
self.canvas.coords(
```

```
    self.target,
```

```
    self.target_x,
```

```
    self.target_y,
```

```
    self.target_x + 25,
```

```
    self.target_y + 25
```

```
)
```

```
self.check_collision()
```

```
self.root.after(150, self.ai_move)
```

```
def check_collision(self):
```

```
    if (
```

```
        abs(self.player_x - self.target_x) < 30
```

```
        and
```

```
        abs(self.player_y - self.target_y) < 30
```

```
    ):
```

```
self.score += 1
```

```
self.info["text"] = (  
    f"Target caught! Score: {self.score}"  
)
```

```
self.target_x = random.randint(  
    50,  
    WIDTH - 50  
)
```

```
self.target_y = random.randint(  
    50,  
    HEIGHT - 50  
)
```

```
root = tk.Tk()  
game = CatchGame(root)  
root.mainloop()
```